Тема 1: Базы данных и СУБД. Определения, основные функции и классификация

1. Определения Базы Данных (БД)

Единого общепризнанного определения нет, но в лекциях выделяются ключевые трактовки (К. Дейт, Д. Кренке):

- **База данных** — это совокупность данных, организованных по определенным правилам, которая предусматривает общие принципы описания, хранения и манипулирования ими. Она независима от прикладных программ.
- **Определение Кренке (важное для понимания структуры):** БД — это *самодокументированное* собрание *интегрированных* записей.
    - *Самодокументированное:* содержит не только данные, но и описание своей структуры (метаданные, словарь данных).
    - *Интегрированное:* содержит файлы данных, индексы (связи) и метаданные приложений.

Типы данных в БД (по назначению):

1. **Операционные данные (OLTP):** Оперативная обработка транзакций (текущая работа предприятия).
2. **Перманентные данные (OLAP/Data Warehouse):** Аналитическая обработка, хранилища данных для систем поддержки принятия решений (DSS).

2. Основные требования к организации данных

Чтобы система считалась полноценной БД, она должна обеспечивать:

1. **Неизбыточность:** каждое данное в единственном экземпляре.
2. **Целостность:** соответствие данных внутренней логике и правилам.
3. **Безопасность:** защита от искажения и разрушения.
4. **Независимость** данных от программ.
5. **Совместное использование** многими пользователями.
6. **Восстановление** после сбоев.
7. **Эффективность доступа** (быстрый отклик).

3. Система управления базами данных (СУБД)

Определение: СУБД — это приложение (программное обеспечение), обеспечивающее создание, хранение, обновление и поиск информации в базах данных.

Состав системы баз данных (более широкое понятие, чем просто СУБД):

- Сами базы данных (одна или несколько).
- СУБД.
- Аппаратное обеспечение.
- Программное обеспечение (ОС, утилиты).
- **Пользователи:**
    - *Прикладные программисты* (пишут программы).
    - *Конечные пользователи* (работают через интерфейсы/меню).

- *Администраторы БД* (управляют системой).

4. Основные функции СУБД

Это «сердце» ответа про то, что именно делает СУБД:

1. **Управление данными во внешней памяти:** СУБД сама решает, как организовать файлы на диске (пользователю не нужно знать физическую структуру).
2. **Управление буферами оперативной памяти:** Кэширование данных для ускорения работы (собственная система буферизации, часто в обход ОС).
3. **Управление транзакциями:**
   - Транзакция — последовательность операций, рассматриваемая как единое целое («все или ничего»).
   - Обеспечивает целостность БД при многопользовательском доступе.
4. **Журнализация (Write-Ahead Logging):** Ведение журнала изменений для надежности. Позволяет восстановить БД после мягких (сбой ПО/питания) и жестких (сбой диска) сбоев.
5. **Поддержка языков БД:** Обычно это SQL (структурированный язык запросов) для DDL (описание данных) и DML (манипулирование данными).

Основные компоненты внутри СУБД:

- *Ядро* (управление памятью и журнализацией).
- *Процессор языка* (оптимизация запросов).
- *Подсистема времени исполнения* (интерфейс с БД).
- *Сервисные программы* (утилиты).

5. Классификация СУБД

В лекциях классификация дается по трем основным признакам:

А) По модели данных (историческая перспектива):

1. **Дореляционные:**
   - *Инвертированные списки:* файлы с записями + индексы.
   - *Иерархические:* древовидная структура (родитель-потомок).
   - *Сетевые:* граф (узлы и связи), где выборка зависит от физической организации.
2. **Реляционные:** данные в виде таблиц (отношений), связь через значения ключей, строгая теория (Кодд), язык SQL.
3. **Постреляционные:** объектно-ориентированные и др. (подробно не раскрываются в начале).

Б) По архитектуре (расположение):

1. **Локальные:** всё на одном компьютере.
2. **Распределенные:** части СУБД/БД на разных компьютерах.

В) По способу доступа (архитектура приложения):

1. **Файл-серверные:**
   - БД лежит на файловом сервере.
   - Приложение (на клиенте) скачивает таблицы целиком и обрабатывает у себя.

    - *Минусы:* высокая нагрузка на сеть, блокировка таблиц целиком.
2. **Клиент-серверные:**
    - Есть Клиент (Front-end) и Сервер (Back-end/SQL-сервер).
    - Клиент шлет запрос (SQL) -> Сервер вычисляет -> Сервер возвращает только результат.
    - *Плюсы:* разгрузка сети, безопасность, транзакции на сервере.
3. **Встраиваемые:** СУБД как библиотека внутри программы (без отдельного сервера).
4. **Сервисно-ориентированные:** обмен через сообщения/очереди.

Как отвечать на экзамене (краткий план):

1. Начни с определения: **БД** — это организованная, самодокументированная совокупность интегрированных записей. **СУБД** — ПО для управления ею.
2. Перечисли **требования**: целостность, отсутствие избыточности, безопасность.
3. Назови 5 главных **функций СУБД**: управление диском, управление RAM (буферы), транзакции, журнал (восстановление), язык (SQL).
4. дай **классификацию**:
    - Модели: Иерархическая -> Сетевая -> Реляционная.
    - Архитектура: Файл-сервер vs Клиент-сервер.

Тема 2: Семантическое моделирование данных

1. Зачем это нужно? (Проблема реляционной модели)

Реляционная модель (таблицы) хороша для компьютера, но имеет недостатки для человека на этапе проектирования:

- Она «плоская» и не всегда передает смысл (семантику) связей.
- Трудно сразу представить предметную область в виде таблиц.
- Проектирование начинается с выделения объектов реального мира («Сущностей»), а реляционная модель не предлагает инструментов для их разделения.

Поэтому существует **семантическое (инфологическое) моделирование** — это первая стадия проектирования, создание концептуальной схемы, понятной человеку и не зависящей от конкретной СУБД.

2. ER-модель (Entity-Relationship)

Самый известный представитель семантических моделей. Предложена Питером Ченом в 1976 году.
Суть: Модель «Сущность-Связь».

Основные понятия ER-модели:

A. Сущность (Entity) Это реальный или представляемый объект, информация о котором должна сохраняться.

- *Обозначение:* Прямоугольник с именем внутри.
- *Типы сущностей:*
    - **Сильная:** существует сама по себе.

    - **Слабая:** зависит от сильной сущности (обозначается двойным контуром).

Б. Связь (Relationship) Это ассоциация между сущностями.

- *Обозначение:* Ромб (если связь слабая — двойной ромб).
- *Степень связи:* количество участников (обычно двое — бинарная связь).
- **Типы связей (кардинальность):**
    1. **1:1 (Один к одному).**
    2. **1:N (Один ко многим).**
    3. **M:N (Многие ко многим).** *Важно: в лекции отмечено, что именно тип M:N является «истинным» типом связи, так как 1:1 и 1:N часто реализуются просто через внешние ключи, а не как отдельные объекты.*

В. Атрибут (Attribute) Это деталь/свойство, уточняющее сущность или связь.

- *Обозначение:* Овал.
- *Ключ сущности:* минимальный набор атрибутов, уникальный для каждого экземпляра.
- *Типы атрибутов:* простые/составные, обязательные/необязательные.

---

3. Правила рисования (Нотация)

На диаграммах:

- **Прямоугольники** — сущности.
- **Ромбы** — связи (соединяются линиями с сущностями).
- **Овалы** — атрибуты (соединяются линией с сущностью).
- **Надписи 1, N, M** возле линий указывают тип связи (один ко многим и т.д.).

---

4. Получение реляционной схемы из ER-схемы (Алгоритм)

Это самая важная практическая часть вопроса: как превратить красивую картинку в таблицы. В лекциях описана пошаговая процедура:

1. **Сущности -> Таблицы.** Каждая простая сущность становится таблицей. Имя сущности = имя таблицы.
2. **Атрибуты -> Столбцы.** Свойства становятся столбцами таблицы.
3. **Идентификаторы -> Первичные ключи (PK).** Уникальный идентификатор сущности становится первичным ключом таблицы.
4. **Связи 1:N и 1:1 -> Внешние ключи (FK).**
    - Связь «превращается» в копию первичного ключа родителя, которая помещается в таблицу-потомка (на сторону «Многие» или «Один»).
    - Пример: Если *Отдел* (1) имеет *Сотрудников* (N), то в таблицу *Сотрудников* добавляем столбец *ID_Отдела*.
5. **Создание индексов.** Создаются индексы для первичных ключей (уникальные) и для внешних ключей (для ускорения соединений).
6. **Обработка подтипов** (если есть наследование):
    - Либо все подтипы в одной общей таблице.

    - Либо для каждого подтипа своя отдельная таблица.

Резюме для ответа:

1. Начни с **проблемы**: таблицы не показывают смысл, нужно промежуточное моделирование.
2. Назови **ER-модель** (Питер Чен, 1976).
3. Опиши **три кита**: **Сущность** (объект), **Связь** (отношение 1:1, 1:N, M:N), **Атрибут** (свойство).
4. Расскажи **алгоритм перехода**: Сущность -> Таблица, Атрибут -> Столбец, ID -> Первичный ключ, Связь -> Внешний ключ.

Часть 1: Структурная часть (Из чего состоит?)

Реляционная модель описывает данные как набор отношений (таблиц).

Основные определения (Стр. 9-10):

1. **Домен:** Это тип данных с определенным смыслом (семантикой). Например, не просто «целое число», а «Возраст сотрудника». Это множество допустимых значений.
2. **Атрибут:** Столбец таблицы. Это пара `<Имя, Домен>`.
3. **Кортеж:** Строка таблицы. Множество пар `<Имя атрибута, Значение>`.
4. **Отношение (Relation):** Это совокупность заголовка (схемы) и тела (набора кортежей). В просторечии — Таблица.

Свойства отношения (важно для экзамена!):

- **Нет дубликатов:** В таблице не может быть двух одинаковых строк (это множество).
- **Неупорядоченность кортежей:** Порядок строк не важен (сверху вниз).
- **Неупорядоченность атрибутов:** Порядок столбцов не важен (слева направо).
- **Атомарность (1НФ):** На пересечении строки и столбца находится только *одно* значение, а не список или массив.

Часть 2: Целостная часть (Какие правила?)

Эта часть отвечает за то, чтобы данные были корректными и связными (Стр. 10-11).

Ключи:

1. **Потенциальный ключ (Candidate Key):** Набор атрибутов, который уникально идентифицирует строку. Он должен быть:
    - *Уникальным.*
    - *Неизбыточным* (если убрать хоть один атрибут, уникальность пропадет).
2. **Первичный ключ (Primary Key):** Один из потенциальных ключей, который мы выбрали главным.
3. **Внешний ключ (Foreign Key):** Атрибут (или набор), который ссылается на Первичный ключ *другой* таблицы.

Два главных правила целостности:

1. **Целостность сущностей (Entity Integrity):** Первичный ключ не может быть NULL (пустым). Мы должны уметь отличить каждую строку.
2. **Ссылочная целостность (Referential Integrity):** Если есть Внешний ключ, то его значение должно либо совпадать с существующим Первичным ключом в родительской таблице, либо быть NULL (если это разрешено). Нельзя ссылаться на несуществующую запись.

Часть 3: Манипуляционная часть (Как работать с данными?)

Здесь описывается, как мы извлекаем и меняем данные. Существует два подхода, которые **эквивалентны** (теорема Кодда):

1. **Реляционная алгебра** (процедурный подход: *как* получить данные, пошаговый рецепт).
2. **Реляционное исчисление** (декларативный подход: *что* мы хотим получить, описание результата).

А. Реляционная алгебра (Стр. 12-14)

Состоит из 8 основных операторов. Результат любой операции — тоже отношение (свойство замкнутости).

4 Теоретико-множественных оператора:

1. **Объединение (UNION):** Складывает строки двух таблиц (A + B).
2. **Пересечение (INTERSECT):** Оставляет только те строки, что есть и там, и там.
3. **Разность (MINUS):** Строки из A, которых нет в B.
4. **Декартово произведение (TIMES):** Каждая строка A соединяется с каждой строкой B (все комбинации).

4 Специальных реляционных оператора:

1. **Ограничение (Restriction / WHERE):** Выбор *строк* по условию (горизонтальный срез).
2. **Проекция (Projection / SELECT):** Выбор *столбцов* (вертикальный срез).
3. **Соединение (JOIN):** Декартово произведение + Ограничение. Самый частый оператор.
4. **Деление (DIVIDE BY):** Самый сложный. Найти такие X, которые связаны со *всеми* значениями Y. (Пример: найти поставщиков, которые поставляют *все* виды деталей).

Б. Исчисление кортежей (Tuple Calculus) (Стр. 18-24)

Это основа языка SQL. Мы не говорим «возьми таблицу, отфильтруй, соедини», мы пишем формулу логики.

- **Переменная кортежа:** Переменная, которая «пробегает» по всем строкам отношения (Range variable).
- **WFF (Правильно построенная формула):** Логическое условие, которому должны соответствовать данные.

Кванторы (суть исчисления):

1. **EXISTS (Существует):** Хотя бы одна строка удовлетворяет условию.

2. **FORALL (Для всех):** Все строки удовлетворяют условию.

Пример логики: «Выбрать Имя (Target list), ГДЕ СУЩЕСТВУЕТ такой Отдел, ЧТО Сотрудник работает в нем И Отдел называется 'IT'».

Резюме для ответа (шпаргалка):

1. **Определение:** Реляционная модель — это теория таблиц (Кодд, Дейт).
2. **Структура:** Отношение (таблица), Кортеж (строка), Атрибут (столбец), Домен (тип). Свойства: нет дублей, нет порядка, атомарность (1НФ).
3. **Целостность:** Первичный ключ (уникальность), Внешний ключ (связь). Правила: PK != Null, FK ссылается на существующий PK.
4. **Манипулирование:**
   - *Алгебра:* 8 операций (Union, Intersect, Minus, Times + Select, Project, Join, Divide). Это «инструкция».
   - *Исчисление:* Логика с кванторами (Exists, ForAll). Это «описание».

Тема 4: Теория проектирования реляционных БД: функциональные зависимости и нормальные формы

1. Функциональные зависимости (ФЗ)

Это фундамент нормализации. Без понимания ФЗ невозможно понять нормальные формы.

Определение (Стр. 30): Пусть `R` — это отношение (таблица), а `X` и `Y` — наборы атрибутов (столбцов). Говорят, что `Y` **функционально зависит от** `X` (записывается `X -> Y`), если каждому значению `X` соответствует в точности одно значение `Y`.

- `X` — детерминант (определитель).
- `Y` — зависимая часть.
- *Пример:* `{Номер паспорта} -> {ФИО}` (У одного паспорта только один владелец).

Аксиомы Армстронга (Правила вывода ФЗ): Чтобы найти все зависимости в таблице, используют правила:

1. **Рефлексивность:** Если `Y` входит в `X`, то `X -> Y`. (Тривиальная зависимость: «ФИО, Возраст» -> «ФИО»).
2. **Дополнение:** Если `X -> Y`, то `XZ -> YZ`.
3. **Транзитивность:** Если `X -> Y` и `Y -> Z`, то `X -> Z`.

Замыкание (Closure):

- Замыкание атрибутов (`X^+`) — это все атрибуты, которые можно определить, зная `X`, используя заданные ФЗ.
- Замыкание множества ФЗ (`S^+`) — это вообще все возможные логические зависимости, которые можно вывести из исходных.

Связь ФЗ и Ключей:

- **Суперключ:** Набор атрибутов, от которого функционально зависят **все** атрибуты таблицы.
- **Потенциальный ключ:** Это *минимальный* суперключ (если убрать любой атрибут, он перестанет быть уникальным).

---

2. Нормализация и Декомпозиция

Цель проектирования: Избежать **избыточности** данных и **аномалий обновления**.

- *Аномалия вставки:* Нельзя записать данные о сотруднике, пока он не привязан к отделу (если все в одной куче).
- *Аномалия удаления:* Удаляя сотрудника, мы случайно удаляем информацию об отделе.
- *Аномалия обновления:* При смене адреса отдела приходится менять его в 1000 строках сотрудников.

Метод: Декомпозиция (разбиение одной таблицы на несколько). **Главное требование:** Декомпозиция должна быть **без потерь (Lossless Join)**. Это значит, что если мы разобьем таблицу `R` на `R1` и `R2`, а потом соединим их обратно (Natural Join), мы должны получить *в точности* исходную таблицу `R`, без лишних или потерянных строк.

---

3. Нормальные формы (НФ)

Это шаги улучшения структуры таблицы. Каждая следующая форма строже предыдущей.

1. Первая нормальная форма (1НФ)

- **Требование:** Все атрибуты **атомарны**. В одной ячейке — одно значение.
- *Нарушение:* В ячейке «Телефоны» написано «123-45-67, 987-65-43».
- *Решение:* Разнести в разные строки или вынести в отдельную таблицу.

2. Вторая нормальная форма (2НФ)

- **Требование:** Таблица в 1НФ **И** каждый *неключевой* атрибут зависит от Первичного Ключа **полностью**, а не от его части.
- *Актуально:* Только для составных ключей (когда ключ состоит из 2+ полей).
- *Нарушение:* Таблица `Поставки` (Ключ: `{Поставщик, Товар}`). Атрибут `Адрес поставщика` зависит только от `Поставщика` (части ключа), а не от `Товара`.
- *Решение:* Вынести `Поставщика` и его `Адрес` в отдельную таблицу.

3. Третья нормальная форма (3НФ)

- **Требование:** Таблица во 2НФ **И** нет **транзитивных зависимостей** неключевых атрибутов от ключа.
- *Суть:* Неключевые поля не должны зависеть друг от друга. «Все поля зависят от ключа, только от ключа и ни от чего кроме ключа».
- *Нарушение:* `Сотрудник` (Ключ: `ID`). Поля: `ID`, `Код_Должности`, `Название_Должности`. `Название` зависит от `Кода`, а `Код` от `ID`. (`ID -> Код -> Название`).
- *Решение:* Вынести справочник должностей отдельно.

4. Нормальная форма Бойса-Кодда (НФБК / BCNF)

- **Требование:** Усиленная 3НФ. **Каждый детерминант** (левая часть любой ФЗ) должен быть **потенциальным ключом**.
- *Проблема 3НФ:* 3НФ пропускает ситуации, когда часть *составного ключа* зависит от *неключевого* атрибута.
- *Пример из лекций:* Стоянка. Ключ составной: `{Номер стоянки, Время}`. Но есть зависимость `Тариф -> Номер стоянки`. `Тариф` — не ключ, но он определяет часть ключа. Это нарушение НФБК.

5. Четвертая нормальная форма (4НФ)

- **Требование:** Таблица в НФБК **И** отсутствуют нетривиальные **многозначные зависимости** (MVD: `X -> Y`).
- *Суть:* Нужно разделять независимые факты.
- *Пример из лекций:* Ресторан делает Пиццу (много видов) и доставляет в Районы (много районов). Виды пиццы не зависят от районов. Хранить это в одной таблице (Ресторан, Пицца, Район) — ошибка, так как придется дублировать все пиццы для всех районов.
- *Решение:* Две таблицы: `{Ресторан, Пицца}` и `{Ресторан, Район}`.

6. Пятая нормальная форма (5НФ)

- **Требование:** Устранение зависимостей соединения (Join Dependency). Таблицу нельзя разбить на меньшие без потерь информации, кроме как на ключи.
- *Суть:* Очень редкий случай, связанный с циклическими ограничениями (Продавец — Фирма — Товар), когда каждый может торговать только определенным товаром определенной фирмы.

7. Шестая нормальная форма (6НФ)

- Упоминается в лекциях для **хронологических (temporal)** баз данных. Требует декомпозиции «до конца», чтобы бороться с избыточностью временных интервалов.

Резюме для ответа:

1. **Функциональная зависимость (`X -> Y`):** Знаю `X` — знаю `Y`.
2. **Ключ:** Минимальный набор атрибутов, от которого зависят все остальные.
3. **1НФ:** Атомарность (одна ячейка = одно значение).
4. **2НФ:** Полная зависимость (нет зависимости от части составного ключа).
5. **3НФ:** Нет транзитивности (неключевые поля не зависят друг от друга).
6. **НФБК:** Любой определитель (детерминант) — это ключ.
7. **4НФ:** Нет многозначных зависимостей (независимые факты разделены).
8. **Декомпозиция:** Должна быть обратимой (без потерь данных).

Тема 5: Теория проектирования хранилищ данных (OLAP)

1. Определения и контекст (Стр. 1)

Чтобы понять хранилища данных (Data Warehouse), нужно противопоставить их операционным базам данных. В лекциях выделяются два «мира» данных:

1. **Операционные данные (OLTP — OnLine Transaction Processing):**
    - Это мир оперативной обработки транзакций.
    - Здесь происходят частые обновления, вставки, изменения.
    - Это текущая работа предприятия (продажи в реальном времени, банковские переводы).
2. **Перманентные данные (OLAP — OnLine Analytical Processing):**
    - Это мир аналитической обработки.
    - Используются в системах поддержки принятия решений (**DSS** — Decision Support System) и системах для руководителей (**EIS**).
    - **Определение:** Базу данных для поддержки принятия решений называют **Хранилищем данных (Data Warehouse)**.
    - Здесь хранятся исторические, неизменяемые данные для анализа.

2. Основные принципы проектирования (Отличия от OLTP) (Стр. 38-39)

Подход к проектированию хранилищ кардинально отличается от проектирования классических реляционных БД. Это различие строится на **нормализации**.

Для OLTP систем (обычные БД):

- *Цель:* Быстрая обработка транзакций и вставка данных.
- *Проблема:* Аномалии обновления (ошибки при частом изменении данных).
- *Решение:* **Нормализация** (разбиение на много мелких таблиц). Это критически важно, чтобы избежать дублирования и ошибок.

Для OLAP систем (Хранилища данных):

- *Цель:* Эффективность выполнения сложных запросов на **выборку (чтение)** огромных объемов данных.
- *Особенность:* Данные загружаются редко (пакетами), но читаются часто.
- *Отношение к нормализации:* В системах OLAP **аномалиями обновления можно пренебречь**, так как данные практически не изменяются пользователями поштучно.
- *Главный принцип построения:* **Денормализация**.
    - Даже если схема базы данных должна быть денормализована по соображениям эффективности (чтобы избежать медленных соединений множества таблиц), разработчик сначала должен понять нормализованную схему, а затем осознанно объединить таблицы для ускорения чтения.
    - Это повышает скорость аналитических отчетов.

⚠ Важное примечание про ETL и ELT

В тексте лекций (стр. 1–97) **отсутствуют** определения процессов ETL и ELT. Скорее всего, это рассказывалось устно или было в пропущенных слайдах.

Однако, чтобы ты не провалился на экзамене, я приведу **стандартные определения** (это база, которую ожидают услышать):

- **ETL (Extract, Transform, Load):** Классический подход.

    1. *Extract (Извлечение):* Забираем данные из источников (OLTP).
    2. *Transform (Преобразование):* Чистим, агрегируем, приводим к нужному формату на отдельном сервере.
    3. *Load (Загрузка):* Кладем готовые «чистые» данные в Хранилище (DWH).
- **ELT (Extract, Load, Transform):** Современный подход (для Big Data/Hadoop).
    1. *Extract:* Забираем данные.
    2. *Load:* Сразу грузим «сырые» данные в мощное хранилище.
    3. *Transform:* Преобразуем данные уже внутри хранилища силами самого хранилища по мере необходимости.

Резюме для ответа (строго по лекциям + логика):

1. **Хранилище данных** — это БД для аналитической обработки (**OLAP**) и поддержки принятия решений (**DSS**).
2. В отличие от **OLTP** (операционных систем), где важна нормализация для избежания аномалий обновления, в хранилищах данные в основном **читаются**.
3. Главный принцип проектирования хранилищ — **Денормализация** ради производительности чтения (уменьшение количества join-ов).
4. *(Дополнительно)* Данные попадают туда через процессы переноса (ETL/ELT).

Тема 6: Транзакции

1. Определение и принцип работы

Транзакция — это последовательность операций, выполняемая как единое целое.

- **Принцип:** «Всё или ничего» (All or nothing).
- Если во время выполнения возникает ошибка, все изменения должны быть отменены (откат), чтобы база данных не осталась в «поломанном» состоянии (например, деньги списались, но не зачислились).

Границы транзакций:

1. **Автоматические (Default):** Каждая отдельная команда SQL (INSERT, UPDATE) — это транзакция. Успех — фиксация, ошибка — откат.
2. **Неявные (Implicit):** Транзакция начинается сама при выполнении команды, но завершить её (COMMIT/ROLLBACK) нужно вручную.
3. **Явные (Explicit):** Мы сами пишем `BEGIN TRANSACTION` (начало) и `COMMIT` (фиксация) или `ROLLBACK` (отмена).
    - Есть также `SAVE TRANSACTION` — создание точки сохранения (savepoint), чтобы откатиться не в самое начало, а к определенному моменту.

2. Свойства транзакций (ACID)

Это "священная корова" баз данных. На экзамене нужно знать расшифровку наизусть:

1. **A — Atomicity (Атомарность):** Транзакция неделима. Либо выполняются все команды внутри неё, либо ни одной.
2. **C — Consistency (Согласованность):** Транзакция переводит БД из одного целостного состояния в другое. Все правила и ограничения (constraints) должны быть соблюдены.
3. **I — Isolation (Изоляция):** Параллельные транзакции не должны мешать друг другу. Промежуточные результаты незавершенной транзакции не должны быть видны другим.
4. **D — Durability (Долговечность/Устойчивость):** Если транзакция зафиксирована (Committed), её результат сохранен навсегда, даже если через секунду выключат свет.

---

3. Управление параллельным выполнением

Когда много пользователей работают одновременно, возникают конфликты. Есть два подхода к управлению (Стр. 54):

1. **Пессимистическое управление:** Использует **блокировки**. «Я меняю строку, поэтому никто другой её не трогает». (Высокая надежность, возможны ожидания).
2. **Оптимистическое управление:** Использует **версионность строк**. «Читайте старую версию, пока я готовлю новую». (Меньше блокировок, но возможны конфликты при сохранении).

4. Неблагоприятные эффекты (Проблемы параллелизма)

Если изоляция настроена плохо, возникают 4 классические проблемы:

1. **Потерянное обновление (Lost Update):** Двое читают одну строку и меняют её. Изменение первого затирается изменением второго.
2. **Грязное чтение (Dirty Read):** Вы читаете данные, которые изменила другая транзакция, но **ещё не зафиксировала**. Если она сделает откат, вы окажетесь с данными, которых никогда не существовало.
3. **Неповторяемое чтение (Non-repeatable Read):** В рамках одной транзакции вы читаете строку дважды. Между чтениями кто-то другой **изменил** (update) или удалил эти данные. Результат: два разных значения в одном запросе.
4. **Фантомное чтение (Phantom Read):** Вы читаете набор строк по условию (например, "все товары дороже 100р"). Между чтениями кто-то **добавил** (insert) новый товар за 150р. Результат: количество строк изменилось (появился «фантом»).

---

5. Уровни изоляции транзакций

Чтобы бороться с этими проблемами, стандарт ANSI SQL (и SQL Server) предлагает уровни изоляции. Чем выше уровень, тем надежнее, но медленнее.

1. **Read Uncommitted (Чтение незафиксированных данных):**
   - *Что делает:* Позволяет читать всё, даже «грязные» данные.
   - *Защита:* Нет.
   - *Плюсы:* Самый быстрый, почти не блокирует.
2. **Read Committed (Чтение зафиксированных данных) — ПО УМОЛЧАНИЮ:**
   - *Что делает:* Читает только то, что уже закоммичено.
   - *Защита:* От «Грязного чтения».

    - *Минус:* Возможно неповторяемое чтение и фантомы.
3. **Repeatable Read (Повторяемое чтение):**
    - *Что делает:* Гарантирует, что если вы прочитали строку, она не изменится до конца вашей транзакции.
    - *Защита:* От «Грязного чтения» и «Неповторяемого чтения».
    - *Минус:* Фантомы всё еще могут появляться (можно добавить новую строку).
4. **Snapshot (Снимок):**
    - *Что делает:* Транзакция видит состояние данных на момент своего начала (как на фотографии). Использует версионирование, а не блокировки.
    - *Защита:* От всех проблем, но требует много ресурсов на хранение версий.
5. **Serializable (Сериализуемость):**
    - *Что делает:* Самый строгий уровень. Полная иллюзия того, что транзакции выполняются по очереди.
    - *Защита:* От всех проблем, включая «Фантомы».
    - *Минус:* Самая низкая производительность, много блокировок.

6. Обработка ошибок

В SQL Server для этого используется конструкция `TRY...CATCH`.

- Если ошибка случается в блоке `TRY`, управление переходит в `CATCH`.
- Функция `XACT_STATE()` позволяет проверить, жива ли транзакция (можно ли её закоммитить или только откатить).

Резюме для ответа:

1. **Транзакция** — атомарная единица работы.
2. **ACID** — Атомарность, Согласованность, Изоляция, Долговечность.
3. **Проблемы:** Грязное чтение (некомиченные данные), Неповторяемое чтение (данные изменились), Фантомы (строки добавились).
4. **Уровни изоляции:**
    - *Read Uncommitted* (можно всё).
    - *Read Committed* (стандарт, без грязи).
    - *Repeatable Read* (без изменений строк).
    - *Serializable* (полная изоляция).

Тема 7: Блокировки

1. Определение и назначение (Стр. 58)

Блокировка (Lock) — это механизм, с помощью которого СУБД синхронизирует одновременный доступ нескольких пользователей к одним и тем же данным.

- **Цель:** Защитить данные от изменений другой транзакцией, пока текущая транзакция с ними работает.
- Управляет всем компонент **Диспетчер блокировок (Lock Manager)**.

2. Свойства блокировки

У любой блокировки есть три главные характеристики:

1. **Гранулярность:** Размер блокируемого объекта (строка, страница, таблица).
2. **Режим (Mode):** Тип доступа (чтение, запись, намерение).
3. **Продолжительность:** Как долго она висит (зависит от уровня изоляции).

3. Гранулярность и Иерархия (Стр. 58-59)

Гранулярность — это выбор того, *что именно* блокировать.

- **Низкая гранулярность (Строка/Row):** Блокируем мало данных.
  - *Плюс:* Высокий параллелизм (другие могут работать с соседними строками).
  - *Минус:* Высокие накладные расходы (нужно много памяти для хранения тысяч блокировок).
- **Высокая гранулярность (Таблица/Table):** Блокируем всё сразу.
  - *Плюс:* Мало накладных расходов.
  - *Минус:* Низкий параллелизм (никто не может работать с таблицей).

Иерархия ресурсов: Блокировки могут накладываться на разных уровнях: `База данных` -> `Таблица` -> `Страница` -> `Строка (RID/Key)`

4. Режимы блокировок (Стр. 60-61)

На экзамене важно знать основные типы:

1. **S (Shared) — Совмещаемая:**
   - Для **чтения** (SELECT).
   - *Совместимость:* Другие тоже могут читать (S), но никто не может писать (X).
2. **X (Exclusive) — Монопольная:**
   - Для **изменения** (INSERT, UPDATE, DELETE).
   - *Совместимость:* Полный запрет. Никто не может ни читать, ни писать этот ресурс.
3. **U (Update) — Обновления:**
   - Промежуточный этап. Используется, когда мы читаем данные с намерением их потом изменить (чтобы избежать тупиков). Позволяет другим читать, но не позволяет другим накладывать U или X.
4. **I (Intent) — Блокировки с намерением (IS, IX, SIX):**
   - *Зачем нужны:* Для защиты иерархии.
   - Если транзакция хочет заблокировать *строку* (X), она сначала вешает на *таблицу* блокировку "с намерением" (IX).
   - Это сигнал для других: "Не блокируйте всю таблицу целиком, внутри кто-то работает с конкретными строками".
5. **Sch (Schema) — Блокировки схемы:**
   - Защищают структуру таблицы при DDL операциях (ALTER TABLE).

Эскалация блокировок (Lock Escalation) (Стр. 63): Если транзакция набрала слишком много мелких блокировок (например, >5000 строк), СУБД автоматически заменяет их одной блокировкой таблицы, чтобы освободить память. Это снижает параллелизм, но спасает ресурсы.

---

5. Взаимоблокировки (Deadlocks) (Стр. 64-66)

Определение: Ситуация, когда две (или более) транзакции имеют блокировки на одни ресурсы и пытаются получить блокировки на ресурсы, захваченные друг другом. Возникает **циклическая зависимость**.

- *Пример:* T1 держит Таблицу А и ждет Таблицу Б. T2 держит Таблицу Б и ждет Таблицу А.
- Они будут ждать вечно, пока кто-то не вмешается.

Чем отличается от обычной блокировки? Обычная блокировка — это ожидание, которое закончится (когда другой пользователь закончит работу). Взаимоблокировка сама не разрешится.

Алгоритм обнаружения (Detection):

1. В СУБД есть специальный поток **Lock Monitor**.
2. Он просыпается каждые **5 секунд**.
3. Если находит Deadlock -> он снижает интервал проверки до **100 мс** (ожидая частые тупики).
4. Если тупиков нет -> снова засыпает на 5 сек.

Выбор жертвы (Victim): Когда СУБД находит цикл, она должна убить одну из транзакций (сделать ей `ROLLBACK`), чтобы спасти остальные.

- **Критерий:** Выбирается транзакция, откат которой "дешевле" всего (меньше всего затрат ресурсов).
- Также можно задать приоритет вручную (`SET DEADLOCK_PRIORITY`).
- Жертва получает ошибку **1205**.

Резюме для ответа:

1. **Блокировка** — защита данных от конфликтов.
2. **Свойства:** Гранулярность (строка/таблица), Режим, Длительность.
3. **Основные режимы: S** (читаем все вместе), **X** (пишу один), **U** (планирую писать), **Intent** (флаг на уровне таблицы, что внутри занято).
4. **Взаимоблокировка (Deadlock):** Цикл ожидания.
5. **Решение:** СУБД ищет циклы (раз в 5 сек) и убивает "самую дешевую" транзакцию (жертву).

Тема 8: Журнализация. Архитектура и модели восстановления

1. Типы файлов базы данных (Стр. 67)

База данных SQL Server физически состоит из трех типов файлов:

1. **MDF (Primary Data File):** Главный файл. Содержит данные и указывает на остальные файлы. Обязательно один.
2. **NDF (Secondary Data Files):** Вторичные файлы данных (необязательны, нужны для распределения по дискам).
3. **LDF (Log Data File):** Файл **журнала транзакций**. Содержит запись всех изменений. Обязательно минимум один.

2. Принцип Write-Ahead Logging (WAL) (Стр. 75)

Это **самое главное понятие** в журнализации. **Суть:** Прежде чем записать изменения данных на диск (в файл MDF), система **сначала** гарантированно записывает информацию об этом изменении в журнал (LDF).

- Изменения данных происходят в оперативной памяти (в кэше). Страница памяти с несохраненными изменениями называется **«грязной» (dirty page)**.
- Если выключится свет, данные в памяти пропадут. Но благодаря WAL, при включении сервер прочитает журнал и **восстановит** (Redo) все, что успел записать в лог, но не успел в файл данных.

3. Архитектура журнала

А. Логическая архитектура (LSN) (Стр. 70, 73) Журнал — это бесконечная лента записей.

- **LSN (Log Sequence Number):** Каждая операция имеет уникальный номер. LSN растут последовательно.
- **MinLSN:** Минимальный номер LSN, который нужен для успешного отката активных транзакций.
- **Активный журнал:** Часть лога от MinLSN до конца. Эту часть нельзя удалять, она нужна для целостности.

Б. Физическая архитектура (VLF) (Стр. 71) Файл LDF внутри разбит на кусочки — **Виртуальные файлы журнала (VLF)**.

- Журнал работает **циклически**.
- Когда конец файла достигнут, запись продолжается с начала файла (если там есть свободное место, то есть неактивные VLF, которые были «усечены»).

4. Операции журнала (Зачем он нужен?) (Стр. 67-68)

1. **Откат транзакций (Rollback):** Если пользователь нажал «Отмена» или произошла ошибка.
2. **Восстановление при запуске (Crash Recovery):** После сбоя питания сервер приводит БД в согласованное состояние (накат подтвержденных, откат неподтвержденных).
3. **Восстановление "на момент времени":** Если есть бэкапы журнала, можно вернуть базу в состояние "вчера в 14:03:05".

5. Модели восстановления (Recovery Models) (Стр. 68-69)

Это настройка базы данных, которая определяет, как журнал растет и как его чистить. Это **обязательный вопрос на экзамене**.

1. **FULL (Полная):**
   - *Как пишет:* Пишет абсолютно всё.
   - *Очистка:* Журнал **не очищается сам**. Его нужно вручную бэкапить (Backup Log). После бэкапа неактивная часть усекается.
   - *Плюс:* Можно восстановиться на любую секунду.
   - *Минус:* Журнал может разрастись и забить весь диск, если не делать бэкапы.
2. **SIMPLE (Простая):**
   - *Как пишет:* Пишет всё, но...

- *Очистка:* Журнал **автоматически усекается** (очищается) при достижении контрольной точки (Checkpoint). Место освобождается для новых записей.
- *Плюс:* Не требует администрирования лога, экономит место.
- *Минус:* Нельзя восстановиться на конкретную минуту. При сбое теряются данные с момента последнего полного бэкапа БД.

3. **BULK_LOGGED (С неполным протоколированием):**
   - *Как пишет:* Как FULL, но массовые операции (загрузка миллионов строк, перестройка индексов) пишутся минимально (не каждая строка, а только факты изменения страниц).
   - *Зачем:* Чтобы лог не пух при больших загрузках.

6. Контрольная точка (Checkpoint) (Стр. 73)

Это процесс, который сбрасывает **«грязные» страницы** из памяти на диск (в файл данных).

- Это нужно, чтобы при перезагрузке не пришлось перечитывать журнал с момента "сотворения мира".
- В модели SIMPLE контрольная точка обрезает журнал.

Резюме для ответа:

1. **Файлы:** MDF (данные), LDF (журнал).
2. **Принцип:** WAL (сначала в журнал, потом в базу). Гарантирует ACID.
3. **Устройство:** Логически — поток записей с **LSN**. Физически — циклический файл из **VLF**.
4. **Модели восстановления:**
   - *Simple:* Автоочистка, нет бэкапов лога, потеря данных с последнего бэкапа.
   - *Full:* Ручная очистка бэкапом, восстановление на любой момент.
   - *Bulk-Logged:* Оптимизация для массовых вставок.

Тема 9: Безопасность и Аудит

1. Основные понятия (Стр. 78)

Система безопасности выполняет две главные функции:

1. **Аутентификация (Authentication):** Проверка подлинности. Ответ на вопрос «Кто ты?».
   - Это проверка логина и пароля.
   - *Не путать с идентификацией* (установлением физической личности).
2. **Авторизация (Authorization):** Проверка прав. Ответ на вопрос «Что тебе можно делать?».

2. Архитектура безопасности (Три кита)

Вся защита строится на взаимодействии трех сущностей:

А. Участники (Principals) — "КТО" Это те, кто запрашивает доступ. Они делятся по уровням (Стр. 78):

1. **Уровень Windows:** Пользователи домена или локальные пользователи.
2. **Уровень Сервера:**
   - **Имя входа (Login):** Учетная запись для подключения к самому SQL Server.

    - **Роли сервера:** Группы с правами администратора (например, `sysadmin` — бог сервера, `securityadmin`).
3. **Уровень Базы Данных:**
    - **Пользователь (User):** Учетная запись внутри конкретной БД. *Login* (на сервере) отображается в *User* (в БД).
    - **Роли БД:** Наборы прав. Например, `db_owner` (владелец), `db_datareader` (может всё читать).
    - **Роль `public`:** Специальная роль, в которую входят **все** пользователи.

Б. Защищаемые объекты (Securables) — "ЧТО" Ресурсы, к которым ограничивают доступ. Они образуют иерархию (Стр. 80):

- `Сервер` -> `База данных` -> `Схема` -> `Объект (Таблица, Процедура, Представление)`.

В. Разрешения (Permissions) — "КАК" Связь между Участником и Объектом. Управление идет через три команды (DCL — Data Control Language):

1. **GRANT:** Дать разрешение.
    - Опция `WITH GRANT OPTION` позволяет пользователю передавать это право другим.
2. **DENY:** Запретить разрешение.
    - *Важно:* DENY всегда **сильнее**, чем GRANT. Если ты входишь в группу, где разрешено, но лично тебе запрещено — доступ будет закрыт.
3. **REVOKE:** Отменить ранее выданное или запрещенное право (нейтралитет).

3. Цепочки владения (Ownership Chains) (Стр. 84-86)

Это важная концепция для оптимизации и безопасности. **Суть:** Если один объект (например, Представление View) обращается к другому объекту (Таблице), и у них **один и тот же владелец**, то SQL Server **не проверяет** права на второй объект.

- *Пример:* У меня есть права на `View_Salary`, но нет прав на таблицу `Table_Salary`. Если владелец у них один, я смогу увидеть данные через View.
- Это позволяет давать доступ к данным только через хранимые процедуры, скрывая прямые таблицы.

4. Контекстное переключение (Context Switching) (Стр. 87-88)

Механизм, позволяющий выполнять код от имени другого пользователя. Используется конструкция `EXECUTE AS ...` внутри процедур:

- `CALLER` (по умолчанию): Код выполняется от имени того, кто вызвал процедуру.
- `SELF`: От имени создателя процедуры.
- `OWNER`: От имени текущего владельца процедуры.
- `'User_Name'`: От имени конкретного пользователя.

5. Шифрование (Стр. 92)

SQL Server поддерживает иерархию шифрования:

1. **Сертификаты** (X.509).

2. **Асимметричные ключи** (пара открытый/закрытый ключ).
3. **Симметричные ключи** (один ключ, алгоритмы AES, DES).
4. **TDE (Transparent Data Encryption):** Прозрачное шифрование всей базы на диске (защита от кражи жесткого диска).

6. Аудит и Политики (Policy-Based Management) (Стр. 91)

В SQL Server встроена система управления на основе политик (как "антивирус" для настроек сервера).

- **Цель (Target):** Объект (таблица, БД).
- **Аспект (Facet):** Свойство, которое проверяем (например, "имя начинается с...").
- **Условие (Condition):** Правило ("Имя должно начинаться с 'tbl_'").
- **Политика (Policy):** Инструкция, что делать.

Режимы работы политик:

1. *On demand:* Проверка вручную.
2. *On change (prevent):* Запретить действие, если оно нарушает политику (триггер DDL).
3. *On change (log only):* Разрешить, но записать в журнал нарушение.

Резюме для ответа:

1. **Безопасность** = Аутентификация (Логин) + Авторизация (Права).
2. **Модель:** Участники (Login -> User) + Объекты (Иерархия) + Разрешения (Grant/Deny/Revoke).
3. **Цепочки владения:** Если владелец один, права внутри цепочки не проверяются.
4. **Контекст:** `EXECUTE AS` позволяет временно стать другим пользователем.
5. **Аудит:** Управление через Политики (запрет или логирование нарушений стандартов).

Тема 10: MPP системы и Hadoop. HDFS и MapReduce

1. Когда это нужно? (Стр. 94)

Hadoop — это экосистема для обработки огромных массивов данных (Big Data).

- **Когда использовать:**
  - Данных **очень много** (> терабайтов). Они не влезают на один диск.
  - Данные **неструктурированные** (логи, тексты, картинки).
  - Задачу можно разбить на куски (компонуемость).
- **Когда НЕ использовать:**
  - Мало данных (влезет на одну машину — будет быстрее).
  - Нужен **Real-time** (Hadoop — это пакетная обработка, он медленный на старт).
  - Задачи рекуррентные (где каждый шаг зависит от предыдущего).

2. Архитектура хранения: HDFS (Стр. 94-95)

HDFS (Hadoop Distributed File System) — это файловая система, спроектированная для хранения огромных файлов на множестве недорогих серверов (commodity hardware).

Отличия от обычных ФС:

1. **Огромные блоки:** Файлы бьются на блоки большого размера (по умолчанию 64МБ или 128МБ), чтобы читать их потоком (Streaming read).
2. **Отказоустойчивость:** Железо ломается часто, это норма. Система должна это переживать программно.

Участники кластера (Типы узлов):

1. **NameNode (Мастер):** «Мозг» системы.
   - Хранит только **метаданные**: дерево каталогов и карту «какой файл в каких блоках лежит».
   - Это **SPOF** (Single Point of Failure) — если упадет, кластер встанет.
2. **DataNode (Рабочий):** «Мускулы».
   - Хранит реальные **блоки данных**.
   - Постоянно шлет отчеты (heartbeats) Мастеру: «Я жив, у меня есть блоки А, Б, В».
3. **Secondary NameNode:** (Название обманчиво!).
   - Это **НЕ** резервная копия мастера.
   - Это помощник, который упаковывает логи изменений (edits) в образ файловой системы (fsimage), чтобы NameNode быстрее перезагружалась.

3. Обеспечение отказоустойчивости (Replication)

Главный механизм защиты — **Репликация**.

- Каждый блок данных хранится в **3 экземплярах** (по умолчанию) на разных DataNode.
- **Rack Awareness (Знание стойки):** Система старается класть копии так, чтобы, если отрубится питание целой стойки серверов, данные остались доступны в другой стойке.
- **Процесс записи:** Клиент пишет данные на первую ноду, та передает на вторую, вторая на третью (цепочка/конвейер). Запись считается успешной, если записано минимально необходимое число реплик.

4. Распределенные вычисления: Модель MapReduce (Стр. 97)

Это парадигма программирования: «Разделяй и властвуй». Вычисление состоит из двух шагов:

1. **Map (Отображение):**
   - Параллельная обработка данных. На каждой ноде запускается код, который обрабатывает локальные кусочки данных.
   - Вход: `(Ключ, Значение)`. Выход: промежуточные `(Ключ, Значение)`.
   - *Пример:* Подсчет слов. Map берет строку текста и выдает: `(Привет, 1)`, `(Мир, 1)`.
2. **Shuffle and Sort (Перемешивание):**
   - Система сама собирает все данные с одинаковыми ключами и отправляет их на один Reducer.
3. **Reduce (Свертка):**
   - Агрегация результатов.
   - *Пример:* Получает список `(Привет, [1, 1, 1])` и суммирует: `(Привет, 3)`.

Архитектура запуска задач:

- **JobTracker:** Главный диспетчер. Принимает задачу от клиента и раздает указания узлам. Он старается отдать задачу тому узлу, **где уже лежат данные** (Data Locality).
- **TaskTracker:** Исполнитель на узле. Запускает Map или Reduce.

5. Колоночное хранение (HBase)

В схеме на стр. 94 упомянута **HBase**.

- Это база данных поверх HDFS.
- **Колоночное хранение:** Данные хранятся не строками, а колонками. Это позволяет очень быстро читать конкретный атрибут по всей базе (например, "средний возраст") без чтения лишних данных.

Резюме для ответа:

1. **Hadoop** — для больших данных, дешевого железа и пакетной обработки.
2. **HDFS** — хранение.
    - Файлы бьются на большие блоки.
    - **NameNode** хранит карту, **DataNode** хранит блоки.
    - **Отказоустойчивость** через репликацию (3 копии).
3. **MapReduce** — вычисления.
    - **Map:** Параллельная обработка на местах.
    - **Shuffle:** Сортировка по ключам.
    - **Reduce:** Итоговая свертка.
    - Принцип: «Код едет к данным».

Внимание: В предоставленном файле лекций (стр. 1–97) **нет отдельной главы**, посвященной In-Memory базам данных. Эта тема упоминается лишь вскользь в самом начале.

Тема 11: In-Memory базы данных

1. Определение и суть (Стр. 3)

В классических СУБД (например, SQL Server) данные лежат на диске, а оперативная память используется только как кэш (буфер).

In-Memory Database (IMDB) — это класс СУБД, ориентированный на **постоянное присутствие всей базы данных в оперативной памяти**.

- **Базовая предпосылка (из лекций):** Объем оперативной памяти современных компьютеров настолько велик, что можно не беспокоиться о сложной буферизации (подкачке страниц с диска) и хранить всё в RAM.

2. Преимущества

Главное преимущество вытекает из физики:

1. **Сверхвысокая производительность:**

    - Доступ к оперативной памяти (RAM) занимает наносекунды, а к диску — миллисекунды. Разница в скорости на порядки.
    - Нет затрат времени на операции ввода-вывода (I/O) при чтении.
2. **Упрощение алгоритмов:**
    - Не нужен сложный менеджер буферов (Buffer Manager), который решает, какую страницу вытеснить из памяти на диск.
    - Проще индексы (можно использовать прямые указатели памяти вместо номеров страниц диска).

3. Недостатки

1. **Энергозависимость (Волатильность):**
    - Оперативная память теряет данные при отключении питания.
    - *Решение:* Использование журнала транзакций (WAL) на диске или периодических снимков (Snapshot) на диск для обеспечения свойства Долговечности (Durability).
2. **Стоимость:**
    - 1 ТБ оперативной памяти стоит значительно дороже, чем 1 ТБ SSD диска.
3. **Ограничение объема:**
    - Объем базы данных жестко ограничен размером RAM сервера (хотя современные серверы поддерживают терабайты памяти).

4. Примеры использования

In-Memory базы данных применяются там, где критична скорость реакции (Real-time) или сложная аналитика на лету.

Популярные системы (общеизвестные примеры):

1. **Redis / Memcached:** Часто используются как кэш или хранилище "ключ-значение".
2. **SAP HANA:** Полновесная реляционная In-Memory СУБД для бизнес-аналитики.
3. **Tarantool:** (Российская разработка Mail.ru) — используется для высоких нагрузок.
4. **Microsoft SQL Server (In-Memory OLTP):** В обычном SQL Server есть специальный движок (Hekaton), позволяющий помещать отдельные «горячие» таблицы в память.

Резюме для ответа:

1. **Суть:** Данные хранятся в оперативной памяти (RAM), а не на диске. (Стр. 3).
2. **Плюс:** Скорость (нет медленного дискового I/O).
3. **Минус:** Если выключить свет — данные пропадут (нужен журнал на диске), память дорогая.
4. **Применение:** Кэширование, высоконагруженные системы, аналитика в реальном времени.

Тема 12: Инструкции SQL: DDL, DML, DCL, TCL

Язык SQL (Structured Query Language) делится на 4 подмножества в зависимости от того, чем именно мы управляем.

1. Инструкции языка описания данных (DDL — Data Definition Language)

Эти команды управляют **структурой** базы данных (метаданными), а не самими данными.

- **Назначение:** Создание, изменение и удаление объектов (таблиц, индексов, процедур, пользователей).
- **Особенность:** Обычно они неявно фиксируют транзакцию (нельзя откатить в некоторых старых СУБД, но в SQL Server DDL может быть внутри транзакции).

Основные команды:

- **CREATE** — создать объект.
  - *Пример (стр. 49):* `CREATE TABLE Tab1 (Col1 int ...)`
  - *Пример (стр. 81):* `CREATE LOGIN Carol ...`
- **ALTER** — изменить существующий объект.
  - *Пример (стр. 63):* `ALTER TABLE ... SET (LOCK_ESCALATION = ...)`
- **DROP** — удалить объект навсегда.
  - *Пример (стр. 52):* `DROP PROCEDURE dbo.SaveTranExample`

2. Инструкции языка обработки данных (DML — Data Manipulation Language)

Эти команды работают с **строками** (самими данными). Это то, что используется чаще всего.

Основные команды:

- **SELECT** — выборка данных (чтение).
  - *Пример:* `SELECT * FROM Tab1`
- **INSERT** — вставка новых строк.
  - *Пример (стр. 49):* `INSERT INTO Tab1 VALUES (1, 'aaa')`
- **UPDATE** — обновление существующих данных.
  - *Пример (стр. 50):* `UPDATE titleauthor SET royaltyper = 65`
- **DELETE** — удаление строк.
  - *Пример:* `DELETE FROM Tab1 WHERE id=1`
- **BULK INSERT** (стр. 62) — массовая загрузка данных из файла (специфичная команда для быстрой вставки).

3. Инструкции безопасности (DCL — Data Control Language)

Эти команды управляют **доступом** и правами (подробно разобраны в теме 9, стр. 82-84).

Основные команды:

- **GRANT** — **разрешить** (предоставить право).
  - *Пример:* `GRANT SELECT ON Table1 TO User1`
- **DENY** — **запретить** (явный запрет, который перекрывает разрешения).
  - *Пример:* `DENY EXECUTE ON Proc1 TO User2`
- **REVOKE** — **отозвать** (отменить ранее выданное разрешение или запрет).
  - *Пример:* `REVOKE SELECT ON Table1 FROM User1`

4. Инструкции управления транзакциями (TCL — Transaction Control Language)

Эти команды определяют границы транзакций и управляют их исходом (подробно на стр. 50).

Основные команды:

- **BEGIN TRANSACTION** — начало явной транзакции.
  - Увеличивает счетчик `@@TRANCOUNT` на 1.
- **COMMIT TRANSACTION** (или `COMMIT WORK`) — фиксация изменений.
  - Успешное завершение: данные сохраняются на диск (в журнал). Уменьшает `@@TRANCOUNT`.
- **ROLLBACK TRANSACTION** (или `ROLLBACK WORK`) — откат изменений.
  - Отмена всего, что сделано с момента BEGIN. Возвращает `@@TRANCOUNT` в 0.
- **SAVE TRANSACTION** — создание точки сохранения (Savepoint).
  - Позволяет откатиться не в самое начало, а к промежуточному этапу (частичный откат).

Резюме для ответа (Таблица в голове):

Тип	Расшифровка	Что делает	Команды (Примеры)
DDL	Definition	**Структура** (Таблицы, Процедуры)	`CREATE`, `ALTER`, `DROP`
DML	Manipulation	**Данные** (Строки)	`SELECT`, `INSERT`, `UPDATE`, `DELETE`
DCL	Control	**Права** (Безопасность)	`GRANT`, `DENY`, `REVOKE`
TCL	Transaction	**Транзакции** (Логика)	`BEGIN`, `COMMIT`, `ROLLBACK`, `SAVE`

Тема 13: Объекты базы данных: функции, процедуры, триггеры и курсоры

Это исполняемые объекты базы данных. В отличие от таблиц (которые хранят данные), эти объекты хранят **логику** обработки данных.

1. Хранимые процедуры (Stored Procedures)

Это набор инструкций SQL, который компилируется и сохраняется в базе данных под определенным именем.

- **Упоминание в лекциях:** На стр. 52 приводится пример `CREATE PROCEDURE dbo.SaveTranExample`.
- **Назначение:** Выполнение сложных действий, включающих логику (IF/ELSE), транзакции и обработку ошибок.
- **Особенности:**
  - Могут принимать **входные параметры**.
  - Могут возвращать коды возврата или наборы данных.
  - **Безопасность (стр. 83):** На процедуру дают право `EXECUTE`. Это основа механизма **цепочек владения** (стр. 87): пользователь запускает процедуру, а процедура меняет таблицы, к которым у пользователя нет прямого доступа.

2. Функции (Functions)

Это подпрограммы, которые обязательно возвращают значение.

- **Упоминание в лекциях:** На стр. 83 перечисляются права доступа к функциям. Встречаются встроенные системные функции (например, `@@TRANCOUNT`, `XACT_STATE()`, `fn_dblog()`).
- **Типы функций (Стр. 83):**
  1. **Скалярные (Scalar):** Возвращают одно значение (число, строку). Права: `EXECUTE`, `REFERENCES`.
  2. **Табличные (Table-Valued):** Возвращают целую таблицу (набор строк). С ними можно работать как с обычной таблицей в `SELECT`. Права: `SELECT`, `DELETE`, `INSERT`.
- **Отличие от процедур:** Функции обычно используются *внутри* запросов (например, в `WHERE` или `SELECT`), а процедуры вызываются отдельно командой `EXECUTE`.

3. Триггеры (Triggers)

Это особая разновидность хранимых процедур, которые **запускаются автоматически** при наступлении определенного события. Их нельзя вызвать вручную.

- **Упоминание в лекциях:**
  - Стр. 53: Функция `ERROR_PROCEDURE()` может вернуть имя триггера, в котором произошла ошибка.
  - Стр. 91: Используются в **Политиках (Policy-Based Management)**.
- **Типы триггеров:**
  1. **DML-триггеры:** Срабатывают при изменении данных (`INSERT`, `UPDATE`, `DELETE`). Используются для проверки целостности или автоматического аудита изменений.
  2. **DDL-триггеры (Стр. 91):** Срабатывают при изменении структуры (`CREATE`, `ALTER`, `DROP`). Используются администраторами, чтобы *запретить* опасные изменения (например, удаление таблиц).

4. Курсоры (Cursors)

Это механизм для **построчной** обработки результатов запроса.

- **Упоминание в лекциях:** Стр. 54 ("...параметры параллельного выполнения для курсоров").
- **Зачем нужны:** SQL — это язык множеств (работает со всей таблицей сразу). Если нужно перебрать строки одну за другой (как в цикле `for` в программировании) и сделать с каждой что-то сложное, используют курсор.
- **Минусы:** Обычно работают медленнее, чем обычные SQL-запросы, и могут вызывать проблемы с блокировками, если их неправильно настроить.

Резюме для ответа (Таблица сравнения):

Объект	Как запускается?	Что делает?	Главное отличие
Процедура	Вручную (`EXEC`)	Выполняет логику, транзакции	Может не возвращать значение, меняет состояние БД
Функция	Внутри запроса	Возвращает вычисления	Обязательно возвращает результат (значение или таблицу)

Объект	Как запускается?	Что делает?	Главное отличие
Триггер	Автоматически	Реагирует на события	Нельзя вызвать руками. Срабатывает на Insert/Update/Delete
Курсор	Внутри кода	Перебирает строки	Построчная обработка (цикл по записям)

Эта тема касается производительности. В твоих лекциях информация об этом разбросана: **индексы** упоминаются в определениях (стр. 1) и проектировании (стр. 8), а **сегментирование** (шардинг) — в теме про Hadoop (стр. 94).

Тема 14: Оптимизация запроса: индексы, партиционирование, сегментирование

1. Индексы (Indexes)

Это самый базовый и эффективный способ ускорения выборки данных (чтения).

- **Определение (Стр. 1):** Индексы — это структуры данных, которые представляют связи между данными и служат для повышения производительности приложений базы данных.
- **Принцип работы:** Представь книгу. Чтобы найти нужную главу, ты не листаешь все страницы подряд (Full Scan), а смотришь в содержание (Index), находишь номер страницы и открываешь сразу её.
- **Где создавать индексы (Стр. 8, Шаг 5):**
    1. **Первичный ключ (PK):** Для него всегда создается *уникальный* индекс (чтобы быстро проверять уникальность и искать по ID).
    2. **Внешние ключи (FK):** Чтобы ускорить операции соединения (JOIN) таблиц.
    3. **Атрибуты поиска:** Поля, которые часто встречаются в условии `WHERE` (например, Фамилия или Дата).

Блокировки индексов (Стр. 59): В SQL Server блокировки могут накладываться на уровне ключа (Key) или страницы индекса (Page).

- *Плюс:* Быстрое чтение (`SELECT`).
- *Минус:* Медленная запись (`INSERT`, `UPDATE`), так как при каждом изменении данных нужно перестраивать дерево индекса.

2. Партиционирование (Partitioning)

Это разбиение одной большой таблицы на несколько логических частей **в пределах одной базы данных** (обычно на одном сервере).

- **Как это работает:** Таблица физически режется на куски (партиции) по определенному критерию (например, по дате: данные за 2020 год, за 2021 год и т.д.).
- **Зачем нужно:**
    - **Ускорение:** Если запрос нужен только за 2023 год, СУБД будет читать только одну маленькую партицию, а не всю гигантскую таблицу.
    - **Управление:** Можно легко удалить старые данные (архивирование), просто "отстегнув" старую партицию.

- *Упоминание в лекциях:* Косвенно связано с понятием **«Блокировка эскалации» (Стр. 63)**. Иногда блокировку можно настроить так, чтобы она захватывала не всю таблицу, а только партицию (`HOBT` — Heap or B-Tree).

3. Сегментирование (Sharding / Шардинг)

Это распределение данных по **разным физическим серверам**. Это метод горизонтального масштабирования.

- **Отличие от партиционирования:** Партиции лежат на одном сервере (или одном дисковом массиве). Шарды лежат на *разных* компьютерах.
- **Пример из лекций (Hadoop, Стр. 94-96):**
  - В системах Big Data (HDFS) огромные файлы не влезают на одну машину.
  - Файл разбивается на **блоки** (сегменты).
  - Эти блоки разбрасываются по разным узлам кластера (**DataNodes**).
  - **NameNode** хранит информацию о том, где какой кусок лежит (карта шардов).
- **Зачем нужно:**
  - Хранение объемов данных, превышающих емкость одного сервера (Петабайты).
  - Параллельная обработка (MapReduce): вычисления идут одновременно на 100 машинах, каждая обрабатывает свой кусок (шард).

Резюме для ответа (Структура):

1. **Индексы** (уровень одной таблицы):
   - Ускоряют чтение, замедляют запись.
   - Обязательны для PK и FK.
   - Работают как оглавление в книге.
2. **Партиционирование** (уровень одной БД):
   - Разрезание таблицы на части (например, по годам).
   - Ускоряет работу с большими таблицами, упрощает удаление старых данных.
3. **Сегментирование/Шардинг** (уровень множества серверов):
   - Разнос данных по разным физическим машинам.
   - Основа **распределенных систем** (как Hadoop) и MPP. Позволяет масштабироваться бесконечно.

Тема 15: План запроса. Этапы выполнения запроса

1. Кто этим занимается? (Архитектура, Стр. 3)

За обработку твоего SQL-запроса отвечает компонент СУБД, который в лекциях назван **«Процессор языка базы данных»**. Его задачи:

1. Оптимизация запросов на извлечение и изменение данных.
2. Создание исполняемого внутреннего кода (это и есть план).
3. Передача плана в **«Подсистему поддержки времени исполнения»**, которая уже работает с данными.

2. Этапы выполнения запроса

Процесс превращения текста SQL в данные проходит три стадии:

Этап 1: Компиляция и синтаксический анализ (Parsing) СУБД проверяет, правильно ли написан запрос.

- **Синтаксис:** Проверка опечаток (например, `SELLECT` вместо `SELECT`).
- **Упоминание в лекциях (Стр. 49):** Если возникает **ошибка компиляции**, то план выполнения пакета не строится и пакет не выполняется вообще.
- **Блокировки (Стр. 62):** Во время компиляции на таблицы накладывается **блокировка стабильности схемы (Sch-S)**. Это гарантирует, что пока СУБД думает над планом, никто не удалит таблицу или не переименует столбец.

Этап 2: Оптимизация (Optimization) Это самый сложный и «умный» этап.

- **Суть:** Язык SQL — декларативный (мы говорим *что* хотим, но не говорим *как*). СУБД должна сама придумать алгоритм (план).
- **План запроса:** Это последовательность физических операций (какой индекс взять, в каком порядке соединить таблицы — Join Order).
- **Выбор плана (Стр. 64):** Оптимизатор обычно выбирает **лучший** (самый дешевый по ресурсам) план автоматически.
    - *Влияние на план:* Разработчик может вмешаться в этот процесс, используя **подсказки (hints)**, такие как `TABLOCK`, `XLOCK` или указание конкретного индекса. Но в лекциях сказано: «использовать подсказки рекомендуется только опытным администраторам в крайнем случае».

Этап 3: Выполнение (Execution) Сгенерированный план передается на исполнение.

- **Обработчик запросов (Стр. 58):** Определяет, к каким ресурсам нужен доступ, и запрашивает у Диспетчера блокировок нужные права (S, X, U) в зависимости от уровня изоляции транзакции.
- **Ошибка выполнения (Стр. 49):** Если ошибка происходит на этом этапе (например, нарушение уникальности ключа), то отменяется только текущая инструкция (или транзакция целиком, если включен `XACT_ABORT`).

3. Что такое План запроса?

Это инструкция для движка СУБД.

- Если план плохой (например, оптимизатор решил перебрать всю таблицу вместо использования индекса), запрос будет работать медленно.
- В SQL Server планы кэшируются в памяти, чтобы не тратить время на компиляцию одного и того же запроса каждый раз.

Резюме для ответа:

1. **Компонент:** За это отвечает **Процессор языка БД**.
2. **Этап 1 (Компиляция):** Проверка синтаксиса. Накладывается блокировка **Sch-S**. Если ошибка — стоп.

3. **Этап 2 (Оптимизация):** Выбор пути доступа к данным (индексы, join-ы). Результат — **План запроса**.
4. **Этап 3 (Выполнение):** Запрос блокировок, чтение/запись данных. Ошибки здесь вызывают Rollback.
5. **Влияние:** Можно управлять планом через **подсказки (hints)**, но лучше довериться оптимизатору.

Важное предупреждение: В присланном файле лекций (стр. 1–97) **нет прямых упоминаний** терминов "Data Vault" или "Anchor Modeling". Скорее всего, это была отдельная презентация или устный рассказ лектора.

Тема 16: Гибкие методологии проектирования. Data Vault, Anchor Modeling

Эти методологии появились как ответ на проблему классических хранилищ: «Как менять структуру базы данных, не переписывая все старые отчеты и ETL-процессы?».

1. Теоретическая основа: 6-я Нормальная форма (Стр. 46-47)

В лекциях сказано:

- **Определение:** Отношение находится в 6НФ, когда оно **неприводимо**, то есть не может быть декомпозировано дальше без потерь.
- **На практике:** Это означает таблицу, состоящую только из **Первичного ключа** и **Одного атрибута**.
  - *Пример:* Вместо таблицы `Сотрудник (ID, Имя, Зарплата, Адрес)` мы делаем 3 таблицы: `(ID, Имя)`, `(ID, Зарплата)`, `(ID, Адрес)`.
- **Для чего:** Лекции упоминают **хронологические данные (Temporal Data)**. Если зарплата меняется часто, а имя — редко, нет смысла дублировать имя каждый раз при изменении зарплаты. Максимальная декомпозиция решает эту проблему.

Именно на этом принципе строятся гибкие методологии.

---

2. Anchor Modeling (Якорное моделирование)

Это методология, доводящая принципы 6НФ до абсолюта. Она идеально подходит для гибкой разработки (Agile), так как схема БД может эволюционировать без изменения существующих таблиц.

Основные сущности:

1. **Якоря (Anchors):** Таблицы, содержащие только суррогатный ключ (ID). Они определяют *существование* объекта (например, "Клиент"). Больше там ничего нет.
2. **Атрибуты (Attributes):** Таблицы в 6НФ. Содержат `ID Якоря` + `Значение` + `Метаданные (время действия)`.
   - *Принцип:* Для каждого свойства — своя отдельная таблица. Если у Клиента появилось новое свойство "Telegram_ID", мы просто создаем новую таблицу, не трогая старые.
3. **Связи (Ties):** Таблицы, связывающие два и более якоря (аналог M:N).
4. **Узлы (Knots):** Справочники с фиксированными значениями (например, "Пол", "Статус").

Преимущества (почему это Agile?):

- **Неразрушающие изменения:** Добавление новых данных никогда не требует команды `ALTER TABLE` для старых таблиц. Старый код продолжает работать, даже если схема выросла.
- **Обработка истории:** Легко хранить историю изменений каждого отдельного атрибута.

---

3. Data Vault (Дэн Линстедт)

Это гибридный подход, созданный специально для корпоративных хранилищ данных (Enterprise Data Warehouse). Он сочетает надежность 3НФ и гибкость "Звезды".

Архитектура строится на трех типах таблиц:

1. **Хабы (Hubs):** Содержат список уникальных бизнес-ключей.
    - *Суть:* "Кто это?". (Например, Номер автомобиля, Номер накладной).
    - Не содержат описательных данных, только ключ и метаданные (откуда пришло, когда).
2. **Линки (Links):** Связи между Хабами.
    - *Суть:* "Как они связаны?". (Например, Хаб "Водитель" связан с Хабом "Автомобиль").
    - Позволяют связывать данные из разных систем-источников.
3. **Сателлиты (Satellites):** Содержат контекст (атрибуты) и историю изменений.
    - *Суть:* "Характеристики". (Цвет машины, ФИО водителя).
    - Сателлиты привязываются к Хабам или Линкам.
    - *Гибкость:* Если источник данных изменился (добавились поля), мы создаем *новый* Сателлит рядом со старым, не ломая существующую структуру.

Резюме для ответа:

1. **Проблема:** Классические модели (3НФ) сложно менять на лету. Любое изменение требует переделки структуры таблицы.
2. **Решение:** Высокая нормализация (до **6НФ** — см. стр. 46), где данные разбиваются на мелкие независимые части.
3. **Anchor Modeling:**
    - Базируется на 6НФ.
    - **Якорь** (ID) + **Атрибуты** (отдельные таблицы) + **Связи**.
    - Максимально гибкая, позволяет менять схему без боли.
4. **Data Vault:**
    - Стандарт для DWH.
    - **Хаб** (Ключи) + **Линк** (Связи) + **Сателлит** (Данные/Атрибуты).
    - Позволяет легко объединять данные из разных источников и хранить историю.

---

1. Что такое MPP (Massively Parallel Processing)?

MPP (Массово-параллельная архитектура) — это способ построения систем хранения и обработки данных, при котором задача разбивается на множество мелких подзадач, и они выполняются **одновременно** на множестве независимых серверов (узлов).

Главный принцип: Shared Nothing (Ничего общего) В классическом сервере (SMP) все процессоры делят одну память и диски. В MPP системе:

- Есть **кластер** из множества узлов (Nodes).
- У каждого узла **своя** оперативная память (RAM) и **свой** жесткий диск.
- Узлы общаются только через сеть (Interconnect).
- **Данные заранее разрезаны** и разложены по этим узлам (Шардинг/Сегментирование).

Аналогия:

- *Классическая БД:* Один гениальный математик решает задачу с миллионом уравнений.
- *MPP:* Тысяча школьников, каждому дали по 1000 уравнений. Они решают их одновременно, а потом учитель (Master-нода) собирает ответы.

---

2. Распределенное и Колоночное хранение (из вопроса 10)

В вопросе 10 упоминается **«Распределенное и колоночное хранение»**.

1. **Распределенное (Row-oriented vs Distributed):**
    - В MPP данные не лежат в одном месте. Таблица физически разрезана.
    - *Пример из лекций (стр. 94):* HDFS режет файлы на блоки (по 64/128 МБ) и раскидывает их по разным машинам (DataNodes).
2. **Колоночное (Columnar):**
    - В лекциях на схеме (стр. 94) упоминается **HBase**.
    - *Суть:* В обычных БД данные хранятся строками: `[ID=1, Имя=Ivan, Город=Москва]`. Чтобы узнать средний возраст, нужно прочитать всю строку (и имя, и город), что долго.
    - В колоночных БД данные хранятся по столбцам: отдельно файл со всеми Именами, отдельно файл со всеми Городами.
    - *Плюс:* Для аналитики (OLAP) это работает в разы быстрее — мы читаем только нужную колонку, игнорируя остальные.

---

3. Движения данных (Data Movement / Data Shuffle)

Ты спросил про "различные движения". В мире MPP и Big Data это называется **Data Motion** или **Shuffle**. Когда мы делаем сложный запрос (например, JOIN двух таблиц), данные, лежащие на разных серверах, должны "встретиться".

Существует 3 основных типа движения данных:

А. Локальная обработка (Data Locality) — "Идеальный вариант"

Это то, к чему стремится Hadoop (описано на стр. 97 про **JobTracker**).

- **Принцип:** «Код едет к данным».
- Вычисления происходят на том же сервере, где лежат данные. Ничего никуда не пересылается по сети. Это максимально быстро.

Б. Redistribution (Перераспределение / Shuffle)

Это самый тяжелый процесс. Описан в лекциях как этап **Shuffle and Sort** в MapReduce (стр. 97).

- **Сценарий:** Мы хотим сгруппировать данные по ключу (например, посчитать продажи по каждому Городу) или соединить (JOIN) две большие таблицы по ID.
- **Проблема:** Данные по "Москве" могут лежать и на сервере №1, и на сервере №5. Чтобы их просуммировать, их нужно собрать в одном месте.
- **Действие:** Все узлы начинают обмениваться данными через сеть. Все данные с ключом "Москва" летят на узел А, все данные с ключом "СПБ" — на узел Б.
- *В лекциях:* Mapper'ы закончили работу, и система пересылает результаты на Reducer'ы, сортируя их по ключам.

В. Broadcast (Широковещание)

Этот метод часто используется при JOIN большой таблицы с маленькой (справочником).

- **Сценарий:** Огромная таблица "Продажи" (размазана по 100 серверам) соединяется с маленькой таблицей "Валюты" (100 строк).
- **Действие:** Нет смысла гонять терабайты продаж по сети. Мы берем маленькую таблицу "Валюты" и **копируем её целиком** на каждый из 100 серверов.
- Теперь каждый сервер может делать JOIN локально, так как у него есть своя копия справочника.

Г. Gather (Сбор)

- **Сценарий:** Финал запроса (`SELECT ... LIMIT 10` или `COUNT`).
- **Действие:** Все рабочие узлы отправляют свои частичные результаты на один главный узел (Master / Leader), который отдает итог пользователю.

---

4. Модель MapReduce как реализация MPP (Стр. 97)

В лекциях MPP объясняется именно через парадигму MapReduce. Это пошаговое выполнение "движений":

1. **Ввод:** Данные лежат на разных узлах (HDFS блоки).
2. **MAP (Локальная обработка):**
   - Каждый узел обрабатывает свои кусочки данных. Никакой сети, только диск -> процессор.
   - *Результат:* Пары `Ключ: Значение`.
3. **SHUFFLE (Пересылка/Движение):**
   - Самый узкий момент. Магия, когда данные с одинаковыми ключами физически перелетают по сети с разных серверов на один конкретный сервер (Reducer).
4. **REDUCE (Агрегация):**
   - Финальный подсчет на узле, который получил все данные по конкретному ключу.

Резюме для экзамена:

1. **MPP** — это архитектура "Shared Nothing", где много серверов со своими дисками работают над одной задачей параллельно.
2. **Колоночное хранение** — ускоряет аналитику (чтение только нужных атрибутов), в отличие от строкового.

3. **Движения данных (Data Exchange):**
    - **Data Locality:** Обработка там, где лежат данные (идеал, минимум сети).
    - **Shuffle (Redistribution):** Пересылка данных по сети, чтобы собрать одинаковые ключи на одном узле (для JOIN или GROUP BY). Это "тяжелая" операция.
    - **Broadcast:** Копирование маленькой таблицы на все узлы для локального джойна.
4. **MapReduce** — это фреймворк, который автоматизирует эти движения (Map = локально, Shuffle = сеть, Reduce = локально).